THE SCAFFOLD SYSTEM

A layered blueprint for navigating the EdgeSDR-Nexus monorepo

Conceptual design · why it beats reading the repo cold

Why a Scaffold?

A large monorepo is hard to hold in your head — and even harder to hold in a context window.

1. **5 services, 30+ sub-sections**

   frontend, backend, postprocesamiento, edge, and infra — each with its own stack and conventions.

2. **Reading source wastes tokens**

   Answering a simple structural question shouldn't require ingesting 30,000+ lines of code.

3. **Flat docs don't scale**

   One giant README for a multi-service system becomes unreadable and immediately stale.

4. **Repeated re-discovery**

   Without a map, every agent (or engineer) re-explores the same territory from scratch.

CONTEXT COMPRESSED

30,000+

lines of source code

↓ *compresses to*

~2,500

lines of layered markdown

Enough structure to orient. Not so much that it needs its own map.

Three Design Constraints

The scaffold isn't arbitrary — every structural choice traces back to one of these.

01

Finite Context Windows

AI agents can't afford to read everything. Compressing 30,000+ lines into ~2,500 lines of markdown keeps exploration cheap.

02

Structure First, Details Second

Every file follows the same seven-part template — Purpose, Tech Stack, Structure, Entry Points, Interactions, Gotchas, Open Questions — so you always know where to look.

03

Hierarchical, Not Flat

5 services and 30+ sub-sections would be unreadable as one document. Three layers let you zoom in only as far as the task requires.

The Three-Layer Architecture

You zoom in progressively — never straight from zero to source code.

LAYER 0

Master Index

scaffold/INDEX.md — the entire system at a glance

LAYER 1

Section Summaries

scaffold/<section>/main.md — one page per service

LAYER 2

Sub-Section Details

scaffold/<section>/<sub>/main.md — one specific feature

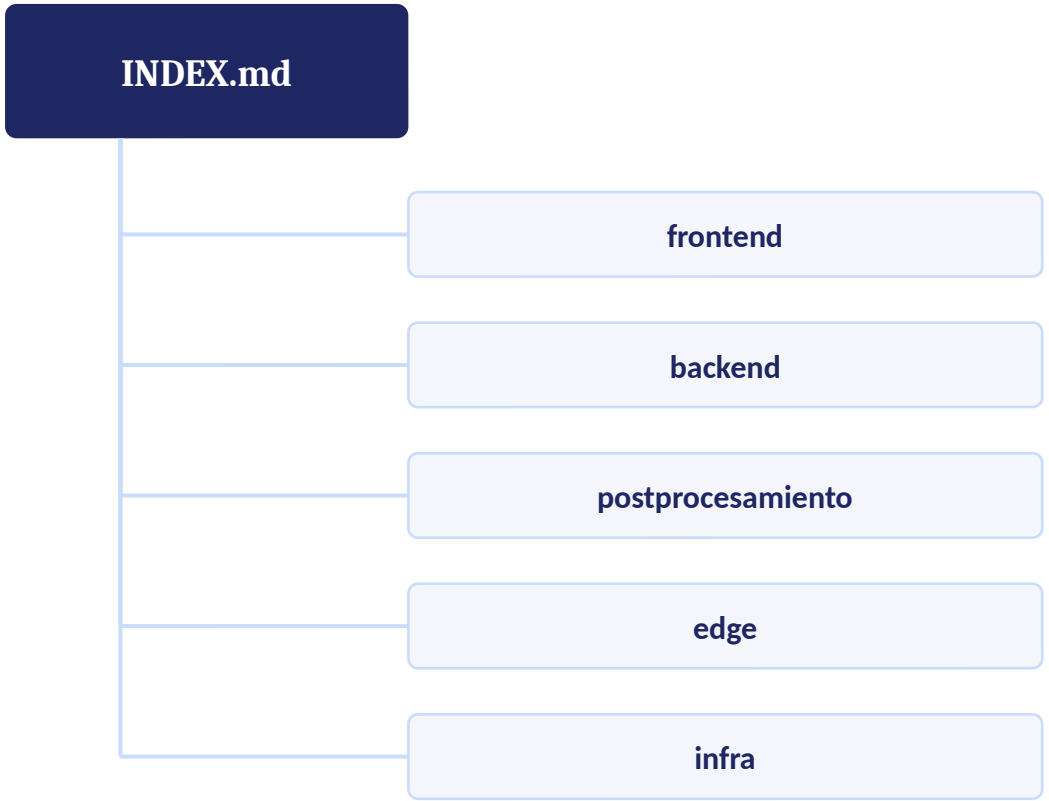
Zoom in only as far as the task requires — reading Layer 1 to fix a Layer 2 bug is wasted context.

Layer 0 — Master Index

scaffold/INDEX.md · read first, always

One page answers: “what services exist, and how do they talk to each other?”

- Lists all 5 sections: frontend, backend, postprocesamiento, edge, infra
- Shows the cross-section data-flow diagram — how services communicate
- Links to key files: docker-compose, AGENTS.md, manifest.json



Layer 1 — Section Summaries

scaffold/<section>/main.md · read before touching any file in that service

Purpose

What the service does, in one sentence

Tech Stack

Languages, frameworks, key libraries

Structure

Directory tree with annotations

Entry Points

Where execution starts

Key Interactions

What calls what — services, DBs, APIs

Gotchas

Known pain points, anti-patterns

Open Questions

Unresolved issues, dead code, TODOs

Same seven fields, every service — scan any section and know exactly where the gotchas live.

Layer 2 — Sub-Section Details

scaffold/<section>/<sub>/main.md · example: the frontend's 7 sub-sections

1

auth

Azure AD SSO, JWT, login/logout

2

monitoring

Real-time spectrum display, configuration

3

campaigns

Campaign lifecycle, data viewing, reports

4

network

Sensor map visualization

5

admin

Antenna and user management

6

audio

WebRTC audio streaming

7

core

App shell, routing, API client

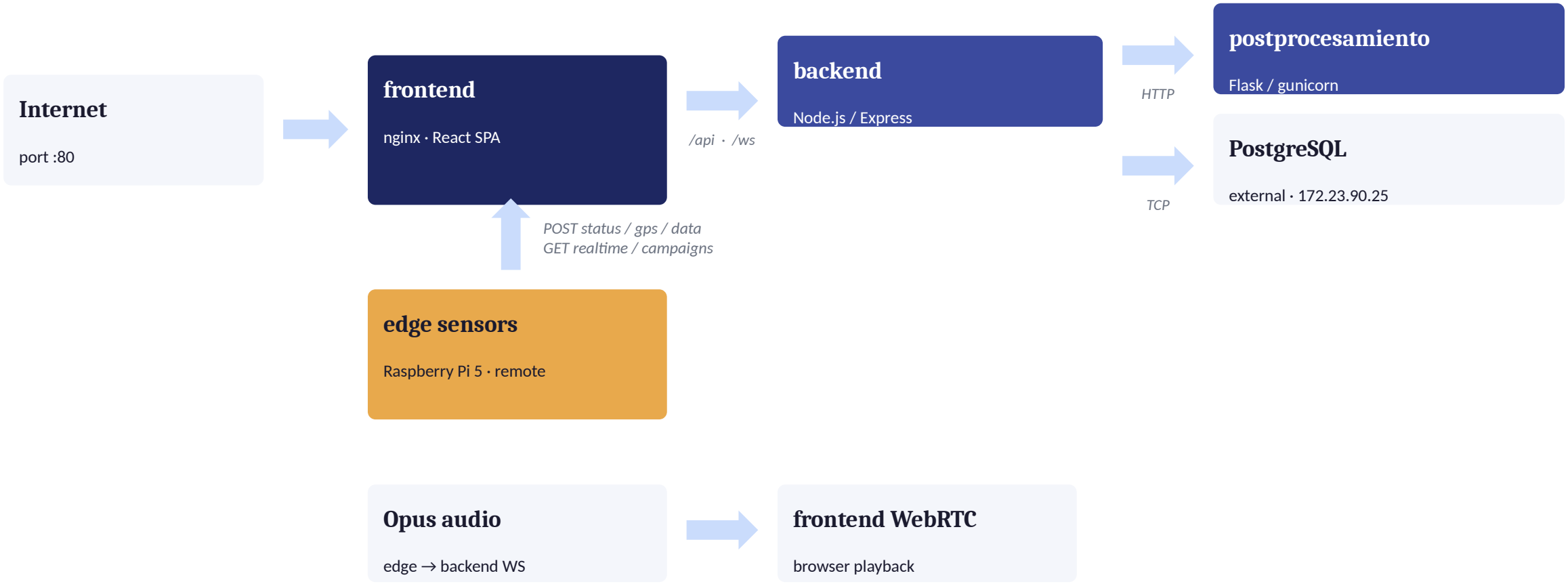
Each Layer 2 file follows the same template, zoomed to a single feature — exactly which files to touch, and which to leave alone.

Navigation Flow

A hard rule from AGENTS.md: never skip straight to grep / find.

Cross-Section Data Flow

The Layer-0 diagram every navigation starts from — who talks to whom.



- Frontend never touches the database directly — always through backend
- Backend calls the Python microservice for spectral analysis
- Edge sensors POST data and receive config via GET

Staying Trustworthy

A map is only useful if you know when to stop trusting it.

manifest.json

Machine-readable navigation for code or AI agents

`path`	file location
`layer`	0, 1, or 2
`parent`	section above it in the hierarchy
`children`	sub-sections below it
`purpose`	one-line description
`last_audited_commit`	git commit when last verified

The Audit Trail

Every file tracks last_audited_commit

- If the scaffold and the code disagree, trust the code — the scaffold is stale
- After fixing a discrepancy, update the scaffold to match reality
- The commit hash lets you diff exactly what changed since the last audit

Why This Design Wins

Compared to a flat README, a wiki, or reading the code cold.

	Traditional Docs	Scaffold System
Token cost	✕ Full source read every time	✓ **~2,500 lines, read once, scoped**
Consistency	✕ Every doc looks different	✓ **One template — always know where to look**
Navigability	✕ Flat, unsearchable wall of text	✓ **3 layers: zoom in only as far as needed**
Programmatic access	✕ None	✓ **manifest.json — code or agents can navigate it**
Freshness signal	✕ Silent rot, no way to tell	✓ **last_audited_commit — staleness is visible**

“The scaffold is not documentation you read once.

It's documentation you navigate every time you work on the codebase.”

INDEX.md → section main.md → sub-section main.md → source code — in that order, every time.